

DSAA2043 Project:
A 2-Approximation Algorithm for the Traveling Salesman Problem
via Minimum Spanning Trees
Deadlines: 5 December at 11:59 p.m. for Part I; 17 December at
11:59 p.m. for others

1 Backgrounds and Definition

The **Traveling Salesman Problem (TSP)** is one of the most fundamental optimization problems in computer science and operations research. Given a set of n points (also called *cities*) in the plane, the objective is to compute a tour that visits every point exactly once and returns to the start point, while minimizing the total length of the tour.

Formally, let $P = \{p_1, p_2, \dots, p_n\}$ be a set of points in the plane. For each point p_i , we use $p_i[x]$ to denote its x -coordinate and $p_i[y]$ as its y -coordinate. Define $d(p_i, p_j)$ as the Euclidean distance between p_i and p_j :

$$d(p_i, p_j) = \sqrt{(p_i[x] - p_j[x])^2 + (p_i[y] - p_j[y])^2}.$$

A *tour* is a permutation π of $[n] = \{1, 2, \dots, n\}$; its cost is

$$\text{cost}(\pi) = \sum_{i=1}^{n-1} d(p_{\pi(i)}, p_{\pi(i+1)}) + d(p_{\pi(n)}, p_{\pi(1)}).$$

The goal is to find a permutation π^* minimizing $\text{cost}(\pi)$. In this project, we consider TSP on points in the **2-dimensional Euclidean plane**. Therefore, the distance function satisfies the **triangle inequality**:

$$d(x, z) \leq d(x, y) + d(y, z) \quad \text{for all } x, y, z \in P.$$

A tour π is called a **c -approximate** solution to TSP if

$$\text{cost}(\pi) \leq c \cdot \text{cost}(\pi^*).$$

A classical result shows that TSP under the triangle inequality has a simple **2-approximation algorithm** using the Minimum Spanning Tree (MST). The goal of this assignment is to design and implement this algorithm. This project consists of two parts:

- **Part 1:** Implement a Minimum Spanning Tree algorithm (Kruskal or Prim).
- **Part 2:** Use the MST to construct a 2-approximate solution to the TSP via an *Euler tour*.

2 Part 1: Minimum Spanning Tree Construction

Given n input points in the plane, you must first construct the complete weighted graph $G = (V, E)$, where

$$V = P, \quad E = \{(p_i, p_j) : 1 \leq i < j \leq n\},$$

and each edge weight is the Euclidean distance between points.

Your task is to build an MST of this graph using one of the following algorithms:

- **Prim's Algorithm**
- **Kruskal's Algorithm**

You may choose either implementation. Efficiency is important: although the graph is conceptualized as complete, you should **avoid explicitly storing all $\Theta(n^2)$ edges in G** if possible. The output of this part is a tree T that spans all n points and has minimum total weight.

3 Part 2: Designing a 2-Approximate Solution to TSP

Once you have computed the MST T , you will use it to generate a 2-approximate TSP tour. The algorithm is based on performing an Euler traversal of the tree.

3.1 Euler Tour of a Tree

Let $T = (V, E)$ be an undirected tree. An *Euler tour* of T is defined as the closed walk produced by performing a depth-first traversal (DFS) of T starting from any chosen vertex $s \in V$. Whenever the DFS moves across an edge $\{u, v\}$ from u to v , we append v to the walk; when the DFS later backtracks from v to u , we append u again. In this traversal, every edge of T is traversed exactly twice — once in each direction. Because DFS returns to the starting vertex s , the resulting walk is a closed cycle. If each undirected edge $\{u, v\}$ is viewed as two directed edges (u, v) and (v, u) , the walk produced by this DFS is an Euler tour of the directed version of the tree: a cycle that visits every directed edge exactly once. Since T spans the entire point set, the Euler tour visits every vertex at least once, although many vertices may appear multiple times.

Example. Consider the following tree T in Figure 1 where s is the starting vertex:

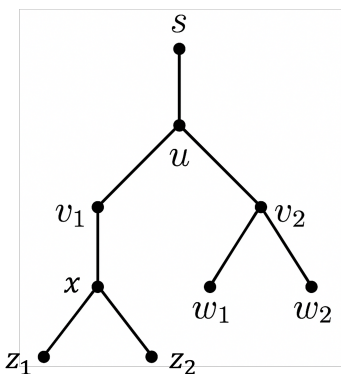


Figure 1: An Example

Starting the DFS at vertex s , one possible Euler tour is:

$s, u, v_1, x, z_1, x, z_2, x, v_1, u, v_2, w_1, v_2, w_2, v_2, u, s.$

Each edge is traversed exactly twice: once going downward during DFS exploration and once returning upward during backtracking. This example illustrates two key properties of Euler tours on trees:

- The walk may visit the same vertex multiple times.
- The total length of the Euler tour contains exactly $2(|V| - 1)$ directed edge traversals.

3.2 Generating a TSP Approximation

Here is a hint about the high-level idea of your algorithm:

1. Find an MST of the complete weighted graph $G = (V, E)$ (see Section 2);
2. Compute an Euler tour of the MST T .
3. Traverse the Euler tour sequence and record the first time each vertex appears. Output the sequence of vertices in the order of their first appearance, then return to the start. You need to ensure that each point (excluding the starting point) is visited exactly once.

3.3 Why is This a 2-Approximation? (Requirement for Your Report)

Let OPT be the length of an optimal TSP tour. It is known that:

- The total weight of all edges in an MST, denoted as $MST(P)$, is a lower bound of OPT : $MST(P) \leq OPT$.
- The Euler tour of the MST has total length exactly $2 \cdot MST(P)$.

You need to prove that the tour produced in your algorithm has a cost of at most $2 \cdot MST(P)$, thus at most $2 \cdot OPT$. You will need to include a short proof of this in your written report.

4 Input and Output Format

4.1 Input Format

The input consists of:

- An integer n (number of points).
- n lines, each containing two floating-point numbers representing (x_i, y_i) coordinates of a point.

Example:

```
5
0.5 1.2
3.7 4.8
1.0 2.0
2.5 3.1
4.2 0.9
```

4.2 Output Format

Your program must output:

- A sequence of n point indices representing the approximate TSP tour.
- The total length of the tour (floating-point).

5 Test Dataset

A test dataset with **10,000 points** is provided separately. You should test the performance of your implementation using this dataset.

6 Submission Instructions

All submissions must be uploaded to Canvas.

6.1 Deadlines and Scores

There are two submission tabs on Canvas. You are required to submit Part I (i.e., your implementation, including source code and a README file) by **5 December at 11:59 p.m.**, which counts for 7 points. You do not need to submit the written report at this stage.

All remaining components must be submitted by **17 December at 11:59 p.m.**, which account for 8 points.

In total, this project is worth 15 points toward your final course grade.

6.2 Source Code

- Submit a single `.py` or `.cpp` file.
- Provide a `README` file explaining how to compile and run your code.
- If you want to use a data structure that has been introduced in this course, you must implement it from scratch. E.g., min-heaps, AVL trees.

6.3 Hand-Written Report (PDF)

You must submit a scanned PDF of your **hand-written report**. The report must include:

- An explanation of data structures and algorithms used (with pseudocode or flowcharts).
- A time complexity analysis of your implementation.
- A short proof explaining why your algorithm is a 2-approximation.
- Your full name, student ID, and email address.

6.4 AI Usage Statement

If you used any AI tools (e.g., ChatGPT, GitHub Copilot), you must upload another PDF (not necessarily hand-written) explaining:

- How you used the tool(s).
- The exact prompt(s) you submitted.